

Clock

↳ It's Periodic signal (square wave)
 ↳ In $\mu c \rightarrow$ Digital circuit $\rightarrow$ sequentially
 CPU timer UART SPI I2C ... etc.

↳ the signal $\rightarrow$ transition from state to another
 ↳ ex CPU $\rightarrow$ fetch $\rightarrow$ decode $\rightarrow$ execute
 ↳ signal $\rightarrow$ clk

clk $\rightarrow$ select operation speed per digital circuit

ex: CPU $\rightarrow$ 8MHz
 ↳ speed instruction $\rightarrow$ 8000000 Instruction per second.

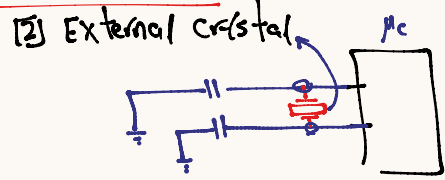
- * Clock source :: who is generate clock
- * Clock Frequency :: which speed for this clk
- * clock Distribution (clock tree)
- * CPU Frequency
- * Peripheral clock

* Clock source

↳ Internal RC oscillator.

↳ in side μc
 ↳ Based R/C
 ↳ Adv
 ↳ Low Price
 ↳ No need external component.
 ↳ Dis Adv
 ↳ low Accuracy
 ↳ Frequency will be affected by
 ↳ temperature
 ↳ voltage
 RC $\rightarrow$ 8MHz $\approx$

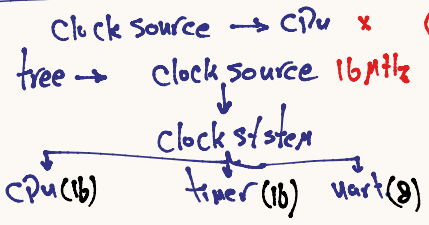
* clock source



3] External clock

4] low Frequency clock

* Clock tree (Dist)



For each Peripheral
 ↳ PreScaler
 ↳ Clock enable

Atmega 32 $\rightarrow$ μc $\rightarrow$ Based AVR Processor

μp	μc
AVR	Atmega 32 Atmega 16
ARM	stm32f103 tinyc
PIC	16f877a
Xtensa	FSP 8206 / FSP 32
RISC-V	FSP 32

* Microchip $\rightarrow$ PIC $\rightarrow$ μc

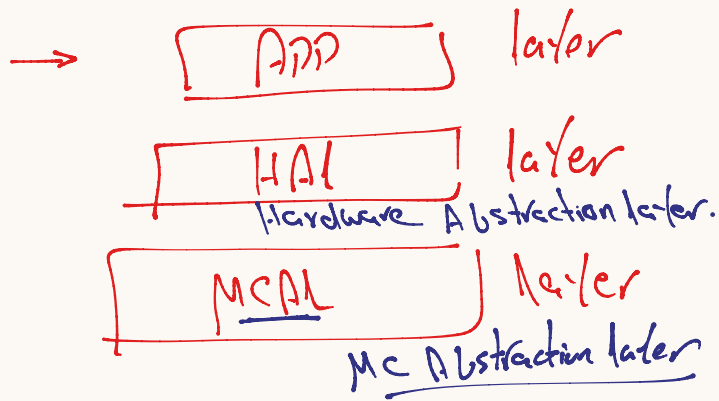
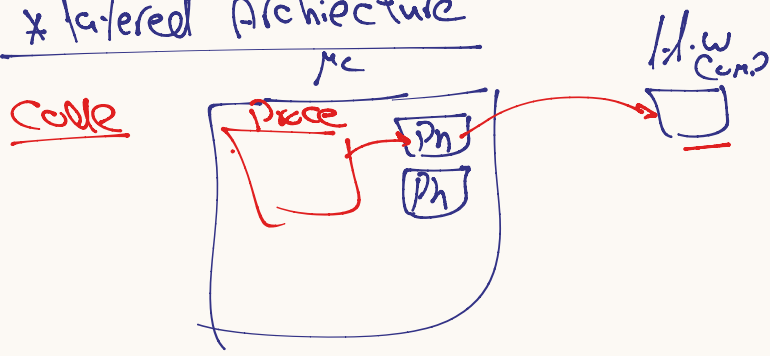
* Atmel $\rightarrow$ AVR $\rightarrow$ μc

Microchip $\rightarrow$ μc Based Pic
 μc Bas AVR

data sheet $\rightarrow$ Atmega 32

Atmega 8
 Atmega 16
 Atmega 128

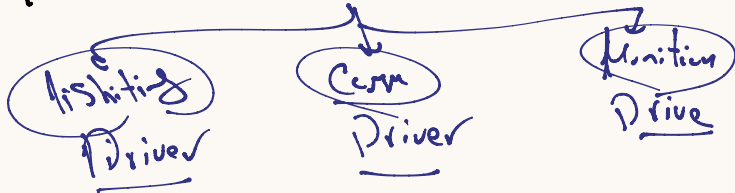
* Layered Architecture



any driver → (lib) for and peripheral
(inside MCU) → must be in MCAL

any driver → (lib) for and H.W component
(outside MCU) → must be in HAL.

any system → logic must be in APP.



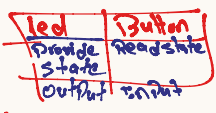
* DIO → Digital Input Output Peripheral

↳ used: to make the processor central on Digital H.W Component

* Digital → (5v, 0v)

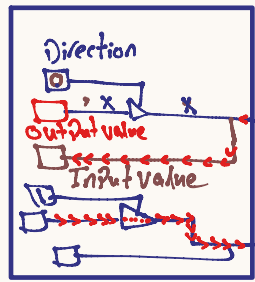
* H.W Component ex:

- ↳ led
- ↳ buzzer
- ↳ digital sensor (IR)
- ↳ button
- ↳ keypad
- ↳ led
- ↳ 7-segment
- ↳ DC motor



* and Peripheral Have (H.W | Memory)

* DIO H.W circuit



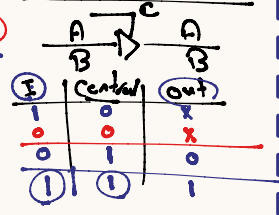
* Direction = 0 → Pin as Input
* Direction = 1 → Pin as output

* Buffer

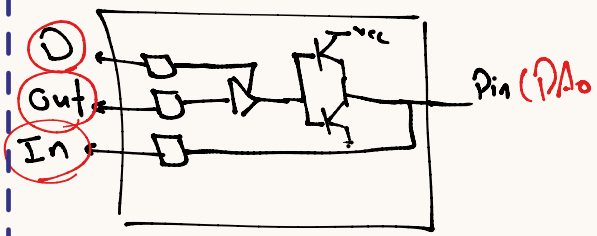
* it's electronics component take input and provide on the output

* Isolate on Embedded

* tri state Buffer



* in AVR → Push Pull output circuit

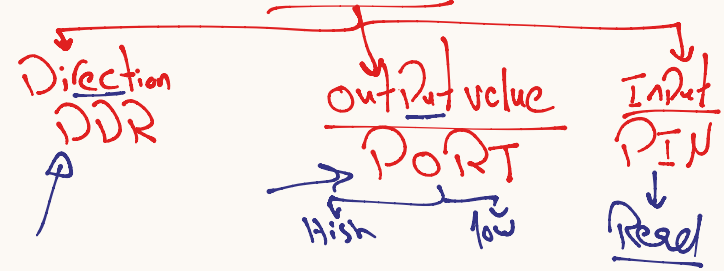


* DIO Memory

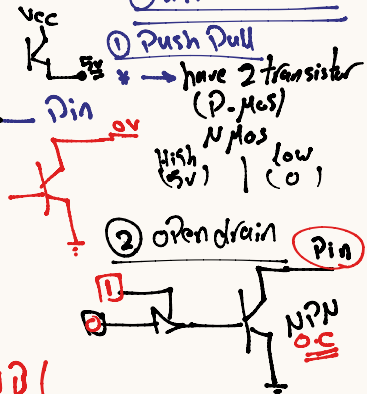
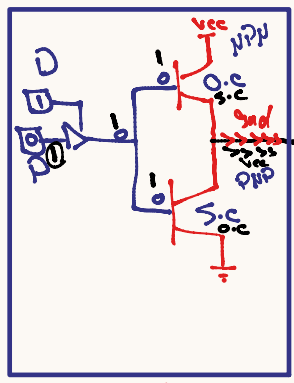
* → 32 pin can work as DIO

* → For each pin to control on it have 3 bit from memory

So we have 3 tags



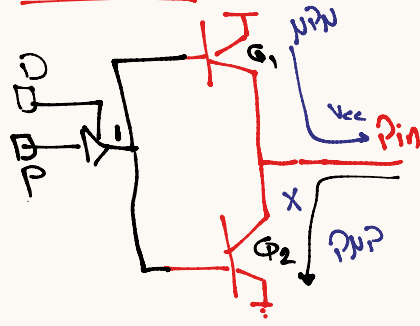
Output circuit



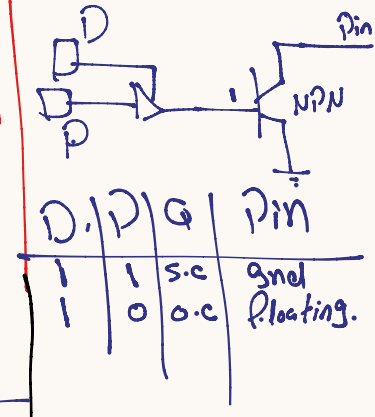
NPN	PNP	NPN	PNP
B=0	B=0	B=1	B=1
O.C	S.C	S.C	O.C

Pin
floating
gnd

Push Pull

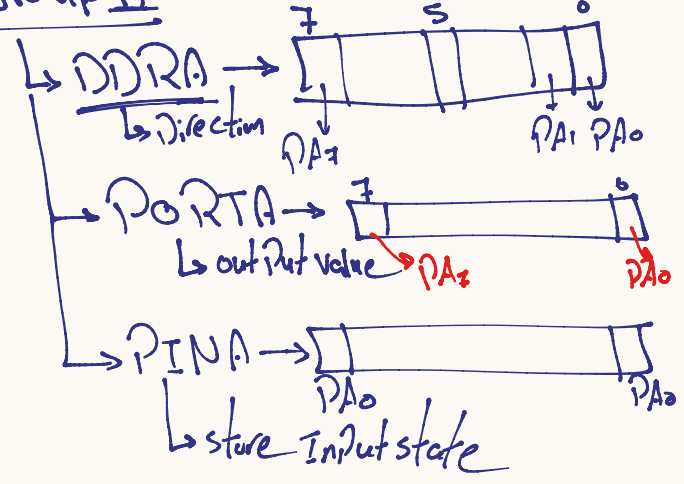


open drain



D	P	G1	G2	Pin
1	1	S.C	O.C	Vcc
1	0	O.C	S.C	gnd

Group A



Group B

DDRB
PORTB
PINB

Group C

DDRC
PORTC
PINC

Group D

DDRD
PORTD
PIND

*Make sure from Address
for this Registers

* Driver → lib

↳ <Module Name>

[
→ Module Name_Private.h
→ Module Name_Interface.h
→ Module Name_Conf.h
→ Module Name_Program.c
]

* DIO, Driver

↳ DIO

↳ DIO_Interface.h
- Private.h
- Conf.h
- Program.c

